Fachbereich II
Computerlinguistik und Digital Humanities

LongNet: Scaling Transformers to 1 Billion Tokens – Critical Assessment of Dilated Attention and an Architectural Review

Term Paper
In Module ML4NLU – Machine Learning for Natural Language Understanding

Lecturer: Prof. Dr. Achim Rettinger
Seminar Tutor: Raghvi Baloni, M.Sc.

vorgelegt von
Bhanu Agarwal (Matrikelnummer: 1805320)
Morteza Abdollahi (Matrikelnummer: 1783075)

Winter Semester 2025/26
Submission on March 24, 2026

Contents

Abstract …… iii
1. Introduction …….. 1
    1.1. The Quadratic Complexity Bottleneck …….. 1
    1.2. Why Long-Range Dependencies Matter for NLU ……. 1
    1.3. Overview of Prior Approaches and Their Failure Modes …….. 1
    1.4. Research Questions and Paper Outline ……..……. 2
2. Background and Related Work ……..…… 3
    2.1. Vanilla Transformer ……..…….. 3
    2.2. Sparse Transformer ……..……... 3
    2.3. Reformer ……..…… 3
    2.4. Longformer ……..…… 4
    2.5. Transformer-XL ……..……. 4
    2.6. Performers and Memory Augmented Models …….. 5
3. LongNet Architecture: Dilated Attention …… 5
    3.1. From Standard to Sparse Attention ……... 5
    3.2. Dilated Attention Mechanism …… 6
    3.3. Multi-Head Offset Strategy …… 6
    3.4. Complexity Derivation and Multi-Scale Mixture …… 6
    3.5. Concrete Configuration and Supporting Components …… 7
4. Evaluation and Experimental Setup …… 7
5. Results and Discussion …… 8
6. Limitations and Future Work …… 9
7. Conclusion …… 10
References …… 11
Appendix …… 12
    Appendix A – Complexity Derivation Summary …… 12
    Appendix B – Comparative Table of Long-Context Architectures …… 12

Abstract

Because standard self-attention grows quadratically with sequence length, current large language models hit a hard practical limit on the context they can process, restricting their applicability to long documents, legal texts, and extended conversations. LongNet addresses this bottleneck by introducing dilated attention, which sparsifies the attention pattern through segmentation and geometrically increasing dilation rates, and then mixes several scales to achieve linear computation O(Nd) and logarithmic token dependency O(log N) while preserving global information flow across the entire sequence. In this term paper, we take a purely theoretical and methodological perspective on LongNet, building on the original paper and related efficient Transformer research. We pay particular attention to how it compares to sparse, local, hash-based, and memory-augmented Transformers, and where its code-focused evaluation leaves open questions for natural language understanding. The analysis addresses LongNet's architectural design, its comparative standing among efficient Transformer variants, its distributed training strategy, and its open questions for natural language understanding. The official LongNet implementation is available at https://aka.ms/LongNet. Supplementary materials for this term paper are available at github.com/bhanu613/ml4nlu-longnet-theoretical-and-architectural-review.

1. Introduction

1.1. The Quadratic Complexity Bottleneck

The work of Vaswani et al. (2017) established self-attention as the core computational mechanism in modern sequence models, and much of current NLP research, from language modeling to question answering, builds directly on top of that design. In the standard formulation, the attention output is computed as:

$O = \text{softmax}(QK^T / \sqrt{d_k})V$, where $Q, K \in \mathbb{R}^{Nxd_k}$, $V \in \mathbb{R}^{Nxd_v}$

The problem is that every token must compare itself against every other token, which means the size of the computation grows as a square of the sequence length- double the tokens, and the cost quadruples. For sequences of a few thousand tokens, this cost is still manageable, but once we move to book-length documents, multi-turn dialogues, or large codebases, the quadratic growth quickly makes training and inference infeasible on standard hardware. Even the most well-known workarounds- Longformer, Sparse Transformer- only manage to push this limit to around 16K or 32K tokens, and those already require specialized setups to run.

From our perspective as computational linguistics students at Universität Trier, this limitation is not just a theoretical inconvenience. This constraint has direct consequences for document-heavy domains such as German legal analysis, lengthy parliamentary records, and multi-chapter historical texts, where artificially segmenting the text into smaller chunks risks losing the cross-segment dependencies that are most important for understanding.

1.2. Why Long-Range Dependencies Matter for NLU

For natural language understanding, the inability to process long contexts is not just an efficiency problem; it is a fundamental constraint on what kinds of reasoning a model can perform. A legal contract, for example, may establish core obligations early on and then qualify, override, or restrict them dozens of pages later; a model that cannot connect these distant passages will fundamentally misread the document. Narrative texts and literary works build characters, themes, and causal chains gradually, often with the most interpretively significant cues separated by chapters. In a long conversation, a user might correct themselves in turn 3, make a promise in turn 40, and ask a follow-up in turn 100- a model that only sees the last few turns has no way of connecting any of these.

We find these examples convincing precisely because they are not edge cases, they are the norm in most serious NLU applications. When context windows are short, models have no choice but to rely on local heuristics or compressed summaries of distant content, which systematically reduces how much of the actual meaning they can represent. This renders context length scaling one of the more consequential open problems in the field, not merely a benchmark concern.

1.3. Overview of Prior Approaches and Their Failure Modes

Several lines of research have tried to escape the quadratic bottleneck, each making a different bet about which part of the full attention matrix can be safely dropped or approximated. The Sparse Transformer (Child et al., 2019) introduces fixed local and

strided patterns, reducing cost to $O(N^{\{\frac{3}{2}\}} \cdot d)$ and making longer sequences tractable but those patterns are designed by hand and are completely independent of the actual content, which means that some token pairs which could matter greatly for understanding simply never interact. The Reformer (Kitaev et al., 2020) tries something smarter: it sorts tokens by how similar they are and groups them into buckets, so attention is only computed between tokens that were likely to interact anyway- this gets the cost down to $O(N \log N \cdot d)$. The mechanism is conceptually appealing, but the approximation it introduces raises a genuine concern which LSH cannot guarantee- semantically related tokens will be placed in the same bucket.

Longformer (Beltagy et al., 2020) chooses a different trade-off: most tokens attend only within a local sliding window, while a small set of designated global tokens attend across the full sequence, keeping complexity at $O(Nd)$. The price is structural, all global information has to be routed through those few global tokens, which can become a bottleneck in complex documents. Transformer-XL (Dai et al., 2019) takes yet another angle, caching the hidden states of previous segments and reusing them as additional context for the current one, which is a useful practical extension but still computes quadratic attention within each segment and can only access distant information in a compressed and increasingly indirect form. Newer approaches like Memorizing Transformers (Wu et al., 2022) and Recurrent Memory Transformer (Bulatov et al., 2022) go further by giving the model a kind of external memory to look up- but this brings its own problems: the retrieval has to be fast, the memory has limited capacity, and compressing old information always risks losing something important. What unites all these approaches is that they each trade away something coverage, exactness, or simplicity to gain scalability, and LongNet is best understood as another point in that same design space, one that tries to preserve complete coverage at the ensemble level while still achieving linear cost.

1.4. Research Questions and Paper Outline

LongNet (Ding et al., 2023) proposes dilated attention as a principled way to achieve linear computational complexity while still allowing information to flow across sequences of up to one billion tokens. Rather than evaluating the model through implementation, we approach it from a theoretical and methodological perspective trying to understand the architecture deeply enough to assess both what it genuinely achieves and where its current limitations lie. As explored during our seminar presentation on LongNet (December 2025, Universität Trier):

1. How does the design of dilated attention and multi-scale segmentation yield $O(Nd)$ computation and $O(\log N)$ token dependency while preserving the model's expressive power?

2. In what ways does LongNet improve upon, or simply differ from, other long-context architectures such as Sparse Transformer, Reformer, Longformer, Transformer-XL, and memory-augmented models and what limitations persist regardless?

3. How does LongNet's distributed training strategy enable scaling to one billion tokens in practice, and what hardware and infrastructure assumptions does that claim rely on?

4. Given that the empirical evaluation focuses exclusively on code modeling, what can reasonably be concluded about LongNet's suitability for long-context natural language understanding tasks such as document-level question answering, legal text analysis, and conversational AI?

The rest of the paper works through these questions in order: Section 2 surveys the efficient Transformer landscape that motivates LongNet, Section 3 analyses the dilated attention architecture in detail, Section 4 examines the experimental setup, Section 5 interprets the results, and Section 6 reflects on limitations and future directions.

2. Background and Related Work

2.1. Vanilla Transformer

The original Transformer architecture (Vaswani et al., 2017) remains the baseline against which all efficient variants are measured. Unlike the RNNs and LSTMs that preceded it, the Transformer processes the whole sequence at once rather than step by step, which is what made it both faster to train and better at capturing long-range patterns in a single pass. Each token attends to every other token through the scaled dot-product attention:

$$O = \text{softmax}(QK^T / \sqrt{d_k})\, V$$

This produces an $N \times N$ attention matrix at each layer, giving $O(N^2 d)$ time complexity and $O(N^2)$ memory. The design is elegant and powerful, but even with memory-efficient implementations such as FlashAttention, the number of attention operations itself does not decrease. This implies that quadratic bottleneck is not just an engineering problem but is a property of the algorithm. It is what every subsequent long-context model we survey in the following subsections tries to work around.

2.2. Sparse Transformer

The Sparse Transformer (Child et al., 2019) reduces the full attention matrix to structured sparse patterns, local blocks combined with a fixed strided pattern bringing complexity down to $O(N^{3/2} \cdot d)$. The idea is straightforward: instead of computing attention between all pairs of tokens, each token only looks at its immediate neighbours plus a few tokens spread out at regular intervals across the sequence- like reading every word carefully nearby, but only every tenth word farther away. The sparsity pattern is content-independent, it is fixed before training and never adapts to the input. The fundamental problem is that the sparsity pattern is decided before seeing any input and never changes regardless of content. In a domain like legal text, where a clause written on page 40 may directly be related to obligation on page 3, a fixed strided pattern cannot confirm whether those specific positions will ever attend to each other. The model has no mechanism to "reach" across the gap which doesn’t align with the predefined stride. LongNet explicitly tries to overcome this by making coverage a structural property of the multi-head ensemble rather than a side effect of the pattern design.

2.3. Reformer

The Reformer (Kitaev et al., 2020) approaches the scalability problem through locality-sensitive hashing. It groups tokens into buckets by their key vectors and computing attention only within each bucket, achieving $O(N \log N \cdot d)$ complexity. The paper also introduces a memory trick for training: instead of saving all intermediate computations

during the forward pass (which normally takes up large amounts of GPU memory), it uses a design that can reconstruct them on the fly. This is not glamorous, but it matters- without it, training on long sequences would run out of memory long before the attention mechanism ever became the bottleneck.

The deeper issue is what the hashing cannot guarantee. Two tokens that are semantically closely related but whose vector representations end up in different buckets will simply never attend to each other, with no fallback mechanism. For tasks where the relevant connection is between a specific phrase on one page and a specific clause on another as is common in legal contracts or technical specifications, this is not a minor approximation error but a fundamental structural gap. LongNet sidesteps this problem by making coverage a guaranteed property of the multi-head design rather than a probabilistic outcome of a hashing procedure.

2.4. Longformer

Longformer (Beltagy et al., 2020) demonstrates that a simple combination of local sliding windows and a small set of globally attending tokens can achieve linear complexity O(Nd) while still performing competitively on long-document benchmarks. Longformer's results on TriviaQA and WikiHop show that this design is not just theoretically clean- it actually works well for the kinds of long-document questions where the relevant evidence could be anywhere in the document. The structural limitation, however, is clear: the global tokens are manually designated and fixed in number, so all long-range communication in the model must pass through that small bottleneck. In a research article or legal document that introduces dozens of named entities, technical concepts, and cross-references, a handful of global tokens cannot realistically carry all the information that would need to flow across the sequence. LongNet's dilated attention is designed so that long-range coverage is distributed across all heads and scales rather than concentrated in a few privileged positions, a direct architectural response to exactly this bottleneck.

2.5. Transformer-XL

Transformer-XL (Dai et al., 2019) extends the effective context by caching hidden states from previous segments. It then incorporates them as additional keys and values for the current segment's attention. This recurrence allows context to span multiple segments without changing the attention mechanism. The mechanism still runs quadratically within each segment. What we found less convincing is what happens over many segments. The further back you go, the more that information has been squeezed and transformed by each layer it passed through- by the time something from ten segments ago might be relevant, it may not be recoverable in its original form anymore. There is no guarantee that the specific details which matter for the current context are still recoverable from those cached states. For tasks where understanding depends on recalling something precise from much earlier in the document, this feels like a real limitation rather than just a theoretical concern. LongNet takes a different approach entirely: rather than trying to propagate past information forward through a chain of compressed states, it structures attention so that long-range positions are always directly reachable within a single forward pass.

2.6. Performers and Memory-Augmented Models

Performers (Choromanski et al., 2021) replace the standard dot-product attention with a kernel approximation using random feature maps. It enables linear-time attention without any sparsity mask. This method is theoretically working well, but its quality depends on the choice of feature maps and the dimensionality of the approximation.

2.6.1 Memorizing Transformers & RMT

Memorizing Transformers (Wu et al., 2022) take a different route: rather than modifying the attention mechanism, they augment it with an external key–value memory. It stores past token representations and can be retrieved via nearest-neighbour search during inference. It is a pragmatic solution, and it works, but it also means the model's performance now depends heavily on the quality of the index and the speed of retrieval. With Recurrent Memory Transformer (Bulatov et al., 2022) we were struck by just how small the actual memory slots are relative to the effective context they are supposed to cover. The idea of reaching a million tokens with a compact learned memory is impressive, but the bottleneck is, very old information must be progressively compressed to make room for newer content.

2.6.2 Synthesis

What all these approaches have in common is that they each give something up to gain something else, exactness for speed, simplicity for coverage, or memory efficiency for retrieval infrastructure. The survey by Tay et al. (2022), tells that every efficient Transformer variant trades away at least one of three properties: coverage completeness, approximation exactness, or simplicity of implementation. Sparse methods sacrifice coverage, hash-based methods sacrifice exactness while recurrent and memory-based methods sacrifice simplicity. Reading it alongside these individual papers was genuinely useful because it helped us see LongNet not as a standalone idea but as one specific answer to a set of trade-offs that the whole field has been working around for years.

3. LongNet Architecture: Dilated Attention

When we read Ding et al. (2023) more carefully, what impressed us most is the consistency with which the dilated attention design is developed, from the theoretical FLOPs analysis through to the distributed training experiments.

3.1. From Standard to Sparse Attention

The starting point is familiar: standard self-attention computes $O = \mathrm{softmax}(QK^T / \sqrt{d_k})\, V$, which requires every query to interact with every key and produces an $N \times N$ attention matrix. What LongNet changes is not the form of this computation but which pairs are actually allowed to participate in it. A filter (mask 1_S) controls which query–key pairs actually, giving:

$$O = \mathrm{softmax}((QK^T / \sqrt{d_k}) \odot 1_S)\, V$$

Earlier sparse models like Sparse Transformer had this same kind of filter, but they chose it based on fixed rules- attend to nearby tokens, skip the rest. LongNet's key difference is that this filter is not fixed arbitrarily- it is built from a dilation pattern that deliberately reaches farther as the sequence gets longer. This means the sparsity is structured and

systematic, not just a way of cutting computation by throwing away arbitrary connections.

3.2. Dilated Attention Mechanism

The input sequence of length N is split into segments of equal length w, giving N/w segments in total. Within each segment, tokens are not all used, instead, every r-th token is sampled according to a dilation rate r. For the i-th segment, the sparsified queries are:

$$\tilde{Q}_i = \left[Q_{\{iw\}}, Q_{\{iw+r\}}, Q_{\{iw+2r\}}, \dots, Q_{\left\{iw+\left\lfloor\left(\frac{w}{r}\right)-1\right\rfloor\cdot r\right\}}\right]$$

and the same sampling is applied to keys and values to form $\tilde{K}_i$ and $\tilde{V}_i$. Attention is then computed independently on each of these smaller, sparsified segments, and the resulting outputs are placed back into their original sequence positions. Think of r as a step size: if you set r=1 you sample every token and get a detailed picture of a short stretch; if you set r=8 you skip most tokens but your view covers eight times the distance.

At first this design seemed strange to us, because it looks like we are deliberately throwing away most of the positions. However, the multi-scale mixture and per-head offset strategy (discussed in Sections 3.3 and 3.4) together ensure that no position is permanently excluded.

3.3. Multi-Head Offset Strategy

One thing we found genuinely clever about the dilated attention design is how it handles the coverage problem, that sparsification might permanently exclude certain positions- is addressed through the multi-head offset strategy. LongNet solves this by giving each head a different starting offset: the j-th head uses $s_j = j \bmod r$, so it samples positions $[iw + s_j, iw + s_j + r, iw + s_j + 2r, \dots]$ within the i-th segment. It is a bit like dividing a long room into sections and assigning each person to inspect different floor tiles- no one person sees everything, but together they cover the whole floor without any tile being missed. When the outputs are concatenated and projected, the model has effectively attended to every position, the coverage is real, not approximate.

3.4. Complexity Derivation and Multi-Scale Mixture

The linear complexity claim is not just asserted, it follows from a straightforward calculation. For a single scale with segment length w_i and dilation rate r_i, each sparsified segment has length w_i/r_i, and attention over it costs roughly $2(w_i/r_i)^2 d$ operations. With N/w_i segments in total, the per-scale cost works out to $2Nw_i d/r_i^2$. Summing across all k scales:

$$\text{FLOPs} = 2Nd \cdot \Sigma_i (w_i / r_i^2)$$

The trick is in choosing w_i and r_i to grow (common ratio $\alpha > 1$) by the same multiplier at each scale. When both grow at the same rate, the cost contribution from each scale actually shrinks geometrically- like the series $(\Sigma_i w_i/r_i^2)$ $1 + 1/2 + 1/4 + \dots$, bounded $((2\alpha/(\alpha-1))$, which adds up to a finite number no matter how many terms you have. That is why adding more scales does not make the total cost blow up: the sum converges, and the overall computation stays proportional to N. The path length result follows a similar logic. After L attention steps, the farthest position reachable grows approximately as $w_0 \cdot \alpha^L / (\alpha-1)$, and setting this equal to N and solving for L gives:

$$L \approx \log_{\alpha} \left[\frac{N(\alpha-1)}{w_0}\right]$$

which is O(log N). As demonstrated during our seminar presentation (December 2025, Universität Trier), the O(Nd) bound becomes intuitive when one traces how the per-segment cost remains fixed while the number of segments grows linearly- the product is always linear in N. The outputs from all scales are finally combined as a weighted mixture $O = \Sigma_i \lambda_i \cdot O|_{\{r_i, w_i\}}$, where λ_i are per-scale weights that allow the model to balance local fine-grained attention against broader long-range coverage depending on what the input requires. The specific parameter values used in the LongNet experiments, and what they imply about design choices, are discussed in the next subsection.

3.5. Concrete Configuration and Supporting Components

In their experiments, Ding et al. (2023) use segment sizes w ∈ {2048, 4096, 8192, 16384, 32768} and dilation rates r ∈ {1, 2, 4, 6, 12}, giving five scales that range from something close to standard local attention at the finest level all the way up to very sparse, wide-ranging patterns at the coarsest. When we looked at these numbers carefully, one thing we noticed is that at the smallest scale, w = 2048 and r = 1, every token within the segment is sampled, meaning this level essentially behaves like ordinary dense attention over short windows. The sparsity and long-range reach only really kick in at the coarser scales. This is a reasonable design choice, but it also means that fine-grained local detail is handled in a fairly conventional way, and the novel contribution mostly lives at the larger scales.

The actual model LongNet uses as its base is called MAGNETO- essentially a Transformer that has been modified to initialize its weights in a more careful way, which helps keep training stable when the model is very large or the sequences are very long. The specific setup used in the experiments has 768 dimensions, 12 attention heads, and 12 layers. For position information, the model uses XPOS encodings, which handle relative positions rather than absolute ones. This matters because a model trained on 32K-token sequences should still work on much longer sequences at inference time — absolute encodings would simply have no learned values for positions they have never seen, while relative ones generalize more naturally. All attention operations, both dilated and the baseline variants in the experiments, are implemented using FlashAttention (Dao et al., 2022). Reading the FlashAttention paper alongside LongNet was genuinely eye-opening for us because it demonstrates that for very long sequences, how memory is accessed. When data moves between GPU memory tiers, it matters as much as the attention formula computation.

4. Evaluation and Experimental Setup

All the experiments in the original paper are run on a code dataset called The Stack- a large collection of open-source code across hundreds of programming languages, chosen because it provides very long files with structured long-range dependencies. To make the comparison fair, all model variants- the standard dense baseline, the sparse baseline, and LongNet's dilated attention- are trained under identical conditions: same architecture size, same tokenizer, same number of training steps, same amount of data per batch. The only thing that changes between them is what happens inside the attention layers. The only

thing that differs between variants is the attention mechanism, which makes the comparison reasonably clean in terms of isolating the effect of dilated attention.

Perplexity on held-out code is the primary quality metric, and sequences of lengths from 2K to 32K tokens are tested. Separately, runtime is measured in a distributed setting for sequence lengths ranging from 8K all the way to 1B tokens, with the batch size scaled down to keep the total token count per batch constant in those extreme-length experiments.

When we thought about this setup, three things stood out as real limitations. First, everything is code. Source code has long-range structure, function definitions, variable scopes, import dependencies, but these are very different from the kinds of dependencies that matter in natural language tasks like legal reasoning, question answering, or coreference resolution. A model that does well on code perplexity has not necessarily learned anything useful for NLU. Second, perplexity measures how well the model predicts the next token on average, but it says nothing directly about whether the model can answer questions, follow instructions, or reason across a document. Without any evaluation on benchmarks like SQuAD or GLUE, there is a real gap between what the paper shows and what we would need to conclude that LongNet is genuinely useful for long-context language understanding. Third, the distributed runtime measurements depend on hardware configurations that are not reproducible without access to large GPU clusters, making it impossible to verify the scalability claims in a typical academic setting.

5. Results and Discussion

The clearest quality result is in Table 2 of the original paper: at 32K tokens, LongNet achieves a test perplexity of 3.01 compared to 3.64 for the Sparse Transformer at matched compute, and the dense Transformer cannot even be evaluated at this length due to its quadratic cost. This is a meaningful gap, and we think it reflects a real advantage of complete multi-head coverage, the Sparse Transformer's fixed patterns simply miss some long-range dependencies that dilated attention can capture.

The runtime results are in some ways even more convincing. With FlashAttention kernels and a distributed setup, dilated attention maintains near-constant inference latency as sequence length grows from 8K all the way to 1B tokens, while dense attention's cost rises steeply and becomes impractical well before the sequence lengths LongNet targets. This is not just a nice-to-have, it is what makes the whole idea of billion-token modeling worth discussing at all.

The FLOPs–perplexity curves show that LongNet gets more out of a given compute budget than a dense Transformer, and the gap widens as FLOPs increase. The most natural explanation is that being able to train on longer sequences gives the model richer and more informative context per training step, so each update does more useful work. Long-context prompting experiments support this directly: as prefix length increases from 2K to 32K tokens, the test loss on held-out suffixes drops consistently, which means the model is actually using the extra context rather than ignoring it. When the authors scale the model from 125M to 2.7B parameters, the improvement in performance follows the same predictable pattern that Kaplan et al. (2020) observed for standard Transformers- which suggests that switching to dilated attention does not break this

fundamental property of how larger models tend to perform better. Reading those scaling laws alongside LongNet's results, we came to see dilated attention less as a radically new idea and more as a way of keeping large-scale training on a viable trajectory without the quadratic cost becoming an obstacle. The GPT-3 results (Brown et al., 2020) made a similar impression on us, much of what looks like emergent capability in large models is really a product of scale and data, and architectures like LongNet matter because they make that scale accessible without exponentially growing compute.

On the distributed training side, one result that deserves attention is that what makes horizontal scaling actually work here is something subtle: when you sparsify the keys and values within each segment, those tensors stay the same size regardless of how long the overall sequence is. So when you add more GPUs to handle a longer sequence, the data each GPU needs to share with the others does not grow- the communication stays manageable even at billion-token scale.

What the paper does not show is just as important. There are no results on standard NLU benchmarks, no SQuAD, no GLUE, no SuperGLUE. All the quality evidence comes from code perplexity on a single domain. For us, working in a small university group without access to large GPU clusters, this also raises a practical point: the impressive scalability in these results depends on distributed infrastructure that most academic groups simply do not have, which makes it hard to reproduce or build on these findings without significant resources.

6. Limitations and Future Work

The most fundamental limitation of LongNet is also the most straightforward one: everything in the paper is about code. Source code has genuine long-range structure, variable scopes, function calls, import chains, but it is far more regular and syntactically predictable than most natural language tasks. Success on code perplexity does not tell us much about whether dilated attention would help with legal document analysis, clinical notes, or multi-document question answering, and right now there is simply no evidence either way. As discussed during our seminar presentation (December 2025, Universität Trier), the transfer from well-structured code to naturalistic language may involve qualitative differences that make the domain gap genuinely uncertain rather than merely unconfirmed.

A second limitation is that perplexity is a weak proxy for actual understanding. Perplexity tells you how surprised the model is by each next token on average- but a model can score well here while still completely failing at tasks that require reasoning, like finding the answer to a question hidden in a document or figuring out what 'it' refers to three paragraphs back. Without evaluation on benchmarks like SQuAD (Rajpurkar et al., 2016), GLUE, or SuperGLUE, the results leave a significant gap between what is shown and what would need to be shown for LongNet to be credibly recommended for NLU applications. Third, the distributed experiments that achieve billion-token scaling require multi-GPU infrastructure and custom FlashAttention kernels that go well beyond what most academic groups can access. This is worth naming directly, because it affects who can actually build on this work. The broader concern here connects to what the scale debate around GPT-3 (Brown et al., 2020) already brought into focus: extremely large

models concentrate research capacity in well-resourced institutions and raise real questions about energy costs and equitable access to this technology.

Future work should start with what is currently missing: systematic evaluation on long-context NLU benchmarks, including document-level question answering, multi-document summarisation, and cross-document coreference. One direction we think is worth exploring is whether you could have the best of both worlds- use dense attention for the tokens immediately around each position (where fine detail matters most) and dilated attention for everything farther away. Right now the current design treats all scales with the same formula, which is clean but probably not the right trade-off for every kind of task. The paper itself points toward genomics and multimodal modelling as promising extensions (Ding et al., 2023), and both seem credible, Genomic sequences are an interesting case- a single gene can span millions of base pairs, and the relevant regulatory signals might be anywhere in that span. Similarly, a research paper that mixes text, figures, and tables is exactly the kind of long, mixed-format input where having a wider context window would actually change what the model understands. Making these architectures viable for smaller research groups, through techniques like parameter-efficient fine-tuning or quantisation of the dilated attention kernels, would also matter for how broadly the ideas can actually be tested and extended.

7. Conclusion

This paper set out to answer four questions about LongNet, and we think the analysis addressed each of them with reasonable clarity.

On the first question, the complexity derivation is solid: the combination of segmentation, geometric dilation schedules, and multi-head offsets does yield linear computation O(Nd) and logarithmic token dependency O(log N), and the math follows cleanly from the geometric series bound. On the second question, LongNet genuinely advances beyond earlier efficient Transformer variants, it preserves complete token coverage at the ensemble level. On the third question, the distributed training strategy is well-designed and the near-constant communication cost is a real advantage, but it also depends on hardware and software infrastructure that most research groups cannot easily access. On the fourth question, the honest answer is that NLU suitability cannot be reliably inferred from code perplexity alone. The theoretical guarantees are real, but whether they translate into better performance on tasks like legal document analysis, question answering, or long-form dialogue remains an open empirical question.

What we came away with is that LongNet is a technically strong and well-motivated contribution to the long-context modeling problem, but one whose practical value for natural language understanding still needs to be demonstrated rather than assumed. Whether dilated attention will ultimately matter for NLU is a question that cannot be answered by this paper- it depends on whether future work runs these experiments on the right tasks and with the right benchmarks. That is the step still missing, and it is the most important one.

References

Beltagy, I., Peters, M. E., & Cohan, A. (2020). Longformer: The long-document Transformer. arXiv:2004.05150.

Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., ... & Amodei, D. (2020). Language models are few-shot learners. In Advances in Neural Information Processing Systems (NeurIPS 2020). arXiv:2005.14165.

Bulatov, A., Kuratov, Y., & Burtsev, M. S. (2022). Recurrent memory transformer. In Advances in Neural Information Processing Systems (NeurIPS 2022). arXiv:2207.06881.

Child, R., Gray, S., Radford, A., & Sutskever, I. (2019). Generating long sequences with sparse transformers. arXiv:1904.10509.

Choromanski, K., Likhosherstov, V., Dohan, D., Song, X., Gane, A., Sarlos, T., ... & Weller, A. (2021). Rethinking attention with Performers. In International Conference on Learning Representations (ICLR 2021). arXiv:2009.14794.

Dai, Z., Yang, Z., Yang, Y., Carbonell, J., Le, Q. V., & Salakhutdinov, R. (2019). Transformer-XL: Attentive language models beyond a fixed-length context. In Proceedings of ACL 2019. arXiv:1901.02860.

Dao, T., Fu, D. Y., Ermon, S., Rudra, A., & Ré, C. (2022). FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In Advances in Neural Information Processing Systems (NeurIPS 2022). arXiv:2205.14135.

Ding, J., Ma, S., Dong, L., Zhang, X., Huang, S., Wang, W., Zheng, N., & Wei, F. (2023). LongNet: Scaling transformers to 1,000,000,000 tokens. arXiv:2307.02486.

Kaplan, J., McCandlish, S., Henighan, T., Brown, T. B., Chess, B., Child, R., ... & Amodei, D. (2020). Scaling laws for neural language models. arXiv:2001.08361.

Kitaev, N., Kaiser, Ł., & Levskaya, A. (2020). Reformer: The efficient Transformer. In International Conference on Learning Representations (ICLR 2020). arXiv:2001.04451.

Rajpurkar, P., Zhang, J., Lopyrev, K., & Liang, P. (2016). SQuAD: 100,000+ questions for machine comprehension of reading. In Proceedings of EMNLP 2016. arXiv:1606.05250.

Tay, Y., Dehghani, M., Bahri, D., & Metzler, D. (2022). Efficient Transformers: A survey. ACM Computing Surveys, 55(6). arXiv:2009.06732.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. In Advances in Neural Information Processing Systems (NeurIPS 2017). arXiv:1706.03762.

Wu, Y., Rabe, M. N., Hutchins, D., & Szegedy, C. (2022). Memorizing Transformers. In International Conference on Learning Representations (ICLR 2022). arXiv:2203.08913.

Appendix

Appendix A – Complexity Derivation Summary

This appendix summarises the key steps in the computational complexity analysis of dilated attention as presented in Ding et al. (2023).

Step 1 — Per-scale cost.
For a single scale with segment length w_i and dilation rate r_i, the sequence of length N is divided into N/w_i segments. Within each segment, tokens are sampled every r_i-th position, leaving approximately w_i/r_i tokens per sparsified segment. The attention computation over one sparsified segment costs: $2(w_i/r_i)^2 d$

Multiplying by the number of segments N/w_i gives the total cost at that scale:

$2Nw_i d/r_i^2$

Step 2 — Summing across scales.
With k scales in total, the overall FLOPs are: FLOPs = $2Nd\ \Sigma_i \frac{w_i}{r_i^2}$

Step 3 — Geometric series bound.
When w_i and r_i are both geometric sequences with common ratio $\alpha > 1$, the ratio w_i/r_i^2 decreases geometrically. The sum therefore converges and is bounded by: $\Sigma_i \left(\frac{w_i}{r_i^2}\right) \leq \left(\frac{2\alpha}{\alpha - 1}\right) w_0$, where w_0 is the smallest segment size. Substituting back gives total FLOPs $\leq (2\alpha)/(\alpha-1)\ w_0 Nd$, which is O(Nd) — linear in sequence length for fixed α and w_0.

Step 4 — Token dependency / path length.
After L attention steps, the maximum distance reachable from any position grows as the sum of receptive fields across scales: $D(L) = w_0\ (\alpha^L - 1)/(\alpha - 1)$

Setting D(L) = N and solving for L: $L \approx log_\alpha \left[\frac{N(\alpha-1)}{w_0}\right]$

For large N the +1 correction term is negligible, giving a maximum path length of O(log N). This confirms that any two tokens in a sequence of length N can influence each other within a number of attention steps that grows only logarithmically with N, even though each individual attention operation is sparse.

Appendix B – Comparative Table of Long-Context Architectures

Model	Complexity	Max Context	Coverage Type	Key Limitation
Vanilla Transformer	$O(N^2 d)$	~2K–8K tokens	Dense, full self-attention	Quadratic cost in sequence length; cannot scale to very long documents in practice.
Sparse Transformer	$O(N^{\{\frac{3}{2}\}} \cdot d)$	~16K–64K tokens	Fixed sparse patterns (local + strided)	Sparsity pattern is content-independent, so some important

				long-range dependencies are never modeled directly.
Reformer	O(N log N · d)	~64K tokens	LSH-based approximate attention	Locality-sensitive hashing can separate semantically related tokens, so attention is only approximate.
Longformer	O(Nd)	~16K–32K tokens	Sliding window + a few global tokens	A small set of global tokens acts as a bottleneck for global information flow in very complex documents.
Transformer-XL	$O(N^2d)$ per segment	~8K–16K tokens per segment	Dense attention within segments + cached segment recurrence	Still quadratic within each segment and depends on compressed segment memories for very long-range information.
Memorizing Transformers	O(Nd) + retrieval cost	~262K tokens	Dense attention + kNN retrieval over external memory	Requires a large external memory and efficient retrieval index; performance depends heavily on memory quality and latency.
Recurrent Memory Transformer (RMT)	O(Nd) per segment	~1M tokens	Attention over current segment + learned recurrent memory slots	Finite memory slots can become an information bottleneck; very old information may be compressed or diluted over time.
LongNet	O(Nd)	~1B tokens	Multi-scale dilated attention with head offsets	Existing evaluation focuses mainly on code; open questions remain about local resolution and performance on NLU tasks.